

Started on Monday, 9 September 2024, 4:34 PM

State Finished

Completed on Monday, 9 September 2024, 4:57 PM

Time taken 23 mins 11 secs

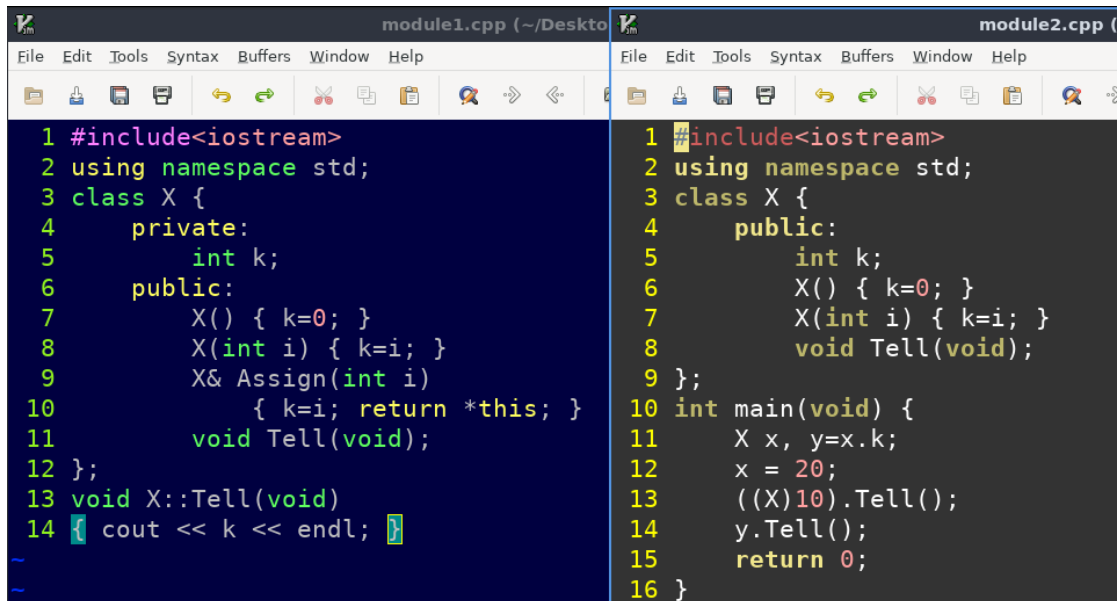
Grade Not yet graded

Question **1**

Correct

Mark 4.00 out of 4.00

Study the following two modules. If you prefer, you may type in the two files with contents as shown. The two C++ modules are to be compiled (independently or together), linked together, and the resulting executable (if any) is to be executed.



```
module1.cpp (~/Desкто)
File Edit Tools Syntax Buffers Window Help
1 #include<iostream>
2 using namespace std;
3 class X {
4     private:
5         int k;
6     public:
7         X() { k=0; }
8         X(int i) { k=i; }
9         X& Assign(int i)
10            { k=i; return *this; }
11         void Tell(void);
12 };
13 void X::Tell(void)
14 { cout << k << endl; }
```

```
module2.cpp (~/Desкто)
File Edit Tools Syntax Buffers Window Help
1 #include<iostream>
2 using namespace std;
3 class X {
4     public:
5         int k;
6         X() { k=0; }
7         X(int i) { k=i; }
8         void Tell(void);
9 };
10 int main(void) {
11     X x, y=x.k;
12     x = 20;
13     ((X)10).Tell();
14     y.Tell();
15     return 0;
16 }
```

Will they compile independently? Will they compile together?

- ☐ They will not compile, there will be "incompatible types" errors in both.
- ☐ They will not compile, because there are syntax errors.
- ☐ They will compile independently but not together. One of them will not compile when they are sought to be compiled together.
- ☒ They will compile independently or together, both ways identically without errors. ✓

Your answer is correct.

The independence between modules regarding types, and our choice of not using a common header to be included, allows this.

The correct answer is:

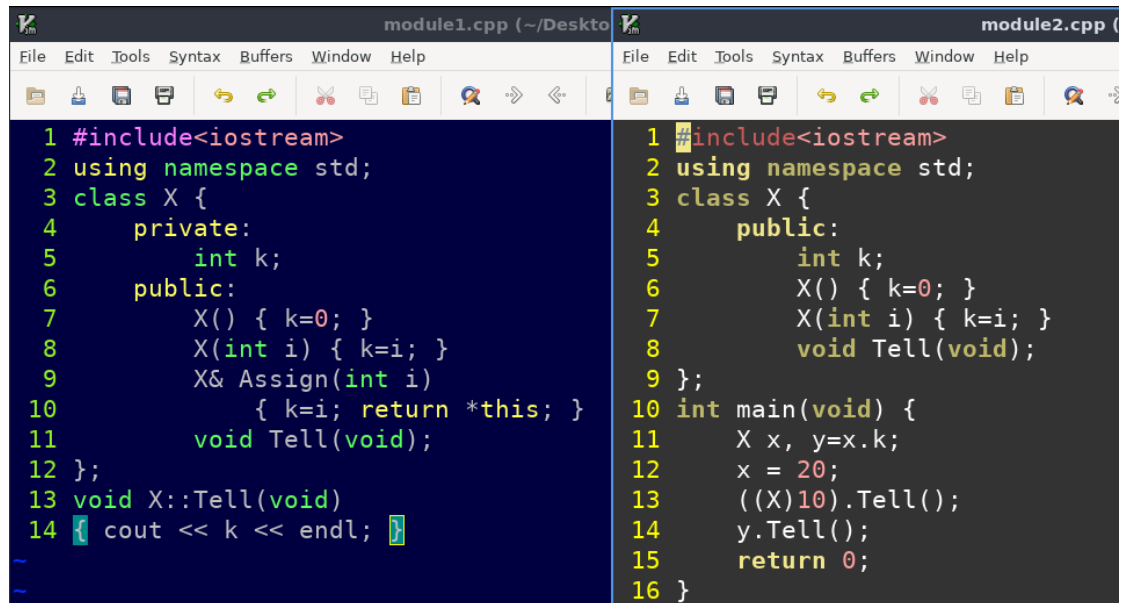
They will compile independently or together, both ways identically without errors.

Question 2

Correct

Mark 4.00 out of 4.00

Study the following two modules. If you prefer, you may type in the two files with contents as shown. The two C++ modules are to be compiled (independently or together), linked together, and the resulting executable (if any) is to be executed.



```
module1.cpp (~/Desktop)
1 #include<iostream>
2 using namespace std;
3 class X {
4     private:
5         int k;
6     public:
7         X() { k=0; }
8         X(int i) { k=i; }
9         X& Assign(int i)
10            { k=i; return *this; }
11         void Tell(void);
12 };
13 void X::Tell(void)
14 { cout << k << endl; }
```

```
module2.cpp (~/Desktop)
1 #include<iostream>
2 using namespace std;
3 class X {
4     public:
5         int k;
6         X() { k=0; }
7         X(int i) { k=i; }
8         void Tell(void);
9 };
10 int main(void) {
11     X x, y=x.k;
12     x = 20;
13     ((X)10).Tell();
14     y.Tell();
15     return 0;
16 }
```

Will they link together?

- ☐ They will not link, there will be "incompatible types" errors in both.
- ☐ They will not compile, because there are syntax errors. The question of linking does not arise.
- ☐ They will compile independently but will not link together. The linker will give "incompatible types" error.
- ☒ They will compile and link together, without errors. ✓

Your answer is correct.

The independence between modules regarding types, and our choice of not using a common header to be included, allows this.

The correct answer is:

They will compile and link together, without errors.

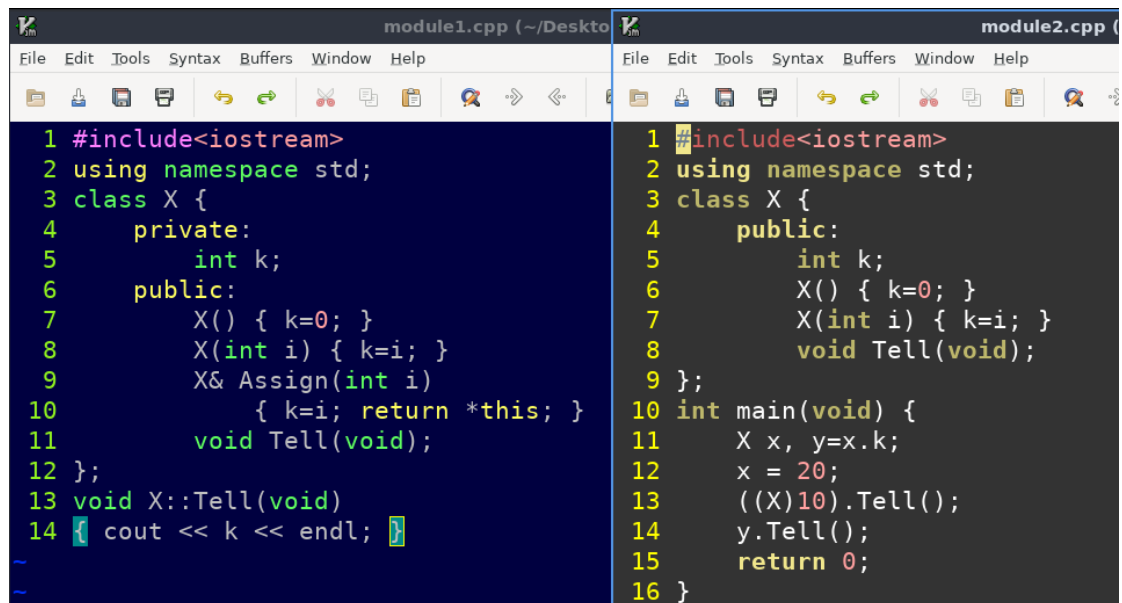
Question 3

Correct

Mark 4.00 out of 4.00

Study the following two modules. If you prefer, you may type in the two files with contents as shown. The two C++ modules are to be compiled (independently or together), linked together, and the resulting executable (if any) is to be executed. The command used to do so is:

```
$ g++ -o xec module1.cpp module2.cpp ; ./xec
```



```
module1.cpp (~/Desktop)
1 #include<iostream>
2 using namespace std;
3 class X {
4     private:
5         int k;
6     public:
7         X() { k=0; }
8         X(int i) { k=i; }
9         X& Assign(int i)
10            { k=i; return *this; }
11         void Tell(void);
12 };
13 void X::Tell(void)
14 { cout << k << endl; }
```

```
module2.cpp (~/Desktop)
1 #include<iostream>
2 using namespace std;
3 class X {
4     public:
5         int k;
6         X() { k=0; }
7         X(int i) { k=i; }
8         void Tell(void);
9 };
10 int main(void) {
11     X x, y=x.k;
12     x = 20;
13     ((X)10).Tell();
14     y.Tell();
15     return 0;
16 }
```

What will be the outcomes?

- ☐ No outcome as they will not compile.
- ☐ No outcome as they will not link.
- ☐ There will be an executable and it will execute but will not produce any output.
- ☒ 10



Your answer is correct.

The independence between modules regarding types, and our choice of not using a common header to be included, allows this.

The correct answer is:

10

Question 4

Correct

Mark 4.00 out of 4.00

Study the following two modules. If you prefer, you may type in the two files with contents as shown. The two C++ modules are to be compiled (independently or together), linked together, and the resulting executable (if any) is to be executed. The command used to do so is:

```
$ g++ -o xec module1.cpp module2.cpp ; ./xec
```

```

module1.cpp (~/Desktop)
1 #include<iostream>
2 using namespace std;
3 class X {
4     private:
5         int k;
6     public:
7         X() { k=0; }
8         X(int i) { k=i; }
9         X& Assign(int i)
10            { k=i; return *this; }
11         void Tell(void);
12 };
13 void X::Tell(void)
14 { cout << k << endl; }

module2.cpp (~/Desktop)
1 #include<iostream>
2 using namespace std;
3 class X {
4     public:
5         int k;
6         X() { k=0; }
7         X(int i) { k=i; }
8         void Tell(void);
9 };
10 int main(void) {
11     X x, y=x.k;
12     x = 20;
13     ((X)10).Tell();
14     y.Tell();
15     return 0;
16 }

```

If we add one more line before "return 0;" in module2.cpp: "y.Tell();" What will be the overall outcomes?

- ☐ No outcome as they will not compile.
- ☐ No outcome as they will not link.
- ☐ There will be an executable and it will execute but will not produce any output.
- ☒ 10
- ☐ 0



Your answer is correct.

The independence between modules regarding types, and our choice of not using a common header to be included, allows this.

The correct answer is:

10

0

Question **5**

Complete

Marked out of
4.00

Explain all the experience of the previous questions. Also explain the implementation of the hidden type example given on the course page.

1. Both moduleone.cpp and moduletwo.cpp compile independently and together because C++ allows certain implicit type conversions that results in no errors during compilation, even though the code might contain unusual constructs. Moreover, there were no syntax errors.

2. The linker successfully resolved all symbol references across the two files. Even though the class X is defined differently in each file, the linker does not check for identical class definitions—only for matching symbols. The Tell() method required by moduletwo.cpp was found in moduleone.cpp, allowing the two files to link together without errors.

3 and 4. I believe the picture accidentally included y.Tell(), otherwise the output would have been just 10. But according to the code shown the output (with y.Tell()) was 10 0. The first Tell() prints 10 from the temporary object created with ((X)10) and the second Tell() prints 0 from y, which was initialized with x.k, where x was default-constructed with k = 0

Hidden Type:

The Class1 class contains a private member, unsigned char *p, which points to a dynamically allocated array. On an x64 system, the class size is 8 bytes, corresponding to the pointer size. The methods setInt() and setFloat() use 4N+1 bytes, while setDouble() uses 8N+1 bytes. However, combining double with other types leads to inconsistent memory usage patterns. A significant issue with the class is that it only retrieves the integer part of float and double values, likely due to incorrect typecasting in its getter functions.

[◀ The missing type](#)[Example1 ▶](#)